



SQL DATA ANALYSIS PROJECT



SCHOOL MANAGEMENT SYSTEM

🎓 BCA STUDENT | LEARNING DATA ANALYTICS

Presented by : Vasudev Sharma

🔧 Tools Used : MYSQL , SQL



YEAR:2026



INTRODUCTION

The School Management System is a SQL-based project designed to manage and analyze school data efficiently. It stores information about students, teachers, courses, classes, and marks in a structured database. The project demonstrates how SQL can be used to retrieve, filter, organize, and analyze data to support better decision-making.

PROJECT OBJECTIVE

- Manage school records efficiently.
- Store student , teacher and course information.
- Analyze academic data using SQL.
- Generate useful reports through SQL queries.

SQL Concepts Used

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- Aggregate Functions
- Subqueries
- JOINS
- LIMIT etc...

PROBLEM STATEMENT

Schools manage a large amount of student, teacher, course, and academic data every day. Managing this information manually can lead to errors, duplicate records, and difficulty in generating reports. This project uses SQL to organize and analyze school data efficiently.

Objectives

- Store student records in a structured database.
- Manage teacher and course information.
- Retrieve data using SQL queries.
- Generate useful reports for analysis.
- Improve data accuracy and organization.

Modules Covered

- Student Management
- Teacher Management
- Course Management

DATABASE OVERVIEW

The School Management System database is designed to store and manage school information in a structured manner. The database consists of multiple related tables that help organize student, teacher, courses records. Relationships between these tables ensure data consistency and make data retrieval efficient.

TABLES NAME	Description
• Student	Stores student information
• Teachers	Stores teacher details
• Courses	Stores Courses information

courses
Course_ID INT
Course_Name VARCHAR(50)
Fees INT
Indexes
PRIMARY

teachers
Teacher_ID INT
Teacher_Name VARCHAR(50)
Subject VARCHAR(50)
Salary INT
Experience INT
Indexes
PRIMARY

student
Student_ID INT
Student_Name VARCHAR(50)
Age INT
Class CHAR(10)
Gender VARCHAR(10)
City VARCHAR(50)
Course_ID INT
Teacher_ID INT
Marks INT
Admission_Date DATE
Indexes
PRIMARY

TABLES



TABLE 1

```
create table student (  
  Student_ID INT PRIMARY KEY,  
  Student_Name VARCHAR(50),  
  Age INT,  
  Class char(10),  
  Gender VARCHAR(10),  
  City VARCHAR(50),  
  Course_ID INT,  
  Teacher_ID INT,  
  Marks INT,  
  Admission_Date DATE);
```

TABLE 2

```
CREATE TABLE Teachers (  
  Teacher_ID INT PRIMARY KEY,  
  Teacher_Name VARCHAR(50),  
  Subject VARCHAR(50),  
  Salary INT,  
  Experience INT);
```

TABLE 3

```
CREATE TABLE Courses (  
  Course_ID INT PRIMARY KEY,  
  Course_Name VARCHAR(50),  
  Fees INT);
```

🔑 **PRIMARY KEY (PK)** : A Primary Key is a column (or set of columns) that uniquely identifies each record in a table.

Characteristics

- Each value must be unique.
- It cannot contain NULL values.
- Every table has only one Primary Key.

TABLES VALUES



TABLE 1

INSERT INTO **Student** VALUES

```
(101, 'Amit', 20 , 'XII', 'Male', 'Delhi', 1 , 201 , 85 , '2025-01-10'),
(102, 'Priya', 21 , 'XII', 'Female', 'Mumbai', 2 , 202 , 92 , '2025-02-15'),
(103, 'Rahul', 19 , 'XII', 'Male', 'Jaipur', 1 , 201 , 78 , '2025-03-01'),
(104, 'Neha', 20 , 'XII' , 'Female', 'Delhi', 3 , 203 , 88 , '2025-01-20'),
(105, 'Karan', 22 , 'XII', 'Male', 'Pune', 2 , 202 , 75 , '2025-04-12'),
(106, 'Riya', 21 , 'XII', 'Female', 'Jaipur', 3 , 203 , 95 , '2025-02-25'),
(107, 'Mohit', 20 , 'XII', 'Male', 'Delhi', 1 , 201 , 69 , '2025-03-18'),
(108, 'Anjali', 22 , 'XII', 'Female', 'Mumbai', 2 , 202 , 81 , '2025-05-05');
```

TABLE 2

INSERT INTO **Teachers** VALUES

```
( 201 , 'Rajesh' , 'SQL' , 50000 , 5 ),
( 202 , 'Sunita' , 'Python' , 60000 , 8 ),
( 203 , 'Anil' , 'Power BI' , 55000 , 6 ),
( 204 , 'Meena' , 'Excel' , 45000 , 4 );
```

TABLE 3

INSERT INTO **Courses** VALUES

```
( 1 , 'SQL' , 12000 ),
( 2 , 'Python' , 25000 ),
( 3 , 'Power BI' , 10000 ),
( 4 , 'Excel' , 5000 );
```

 **FOREIGN KEY (FK) :** A Foreign Key is a column in one table that references the Primary Key of another table. It is used to create a relationship between two tables.

Characteristics

- References the Primary Key of another table.
- Used to create relationships between tables.
- Duplicate values are allowed.

SQL CONCEPT USED

SQL Concepts Used

- **SELECT:**
- **WHERE:**
- **DISTINCT:**
- **AND / OR:**
- **IN:**
- **BETWEEN:**
- **LIKE:**
- **ORDER BY:**
- **Aggregate Functions(COUNT, SUM, AVG, MIN, MAX):** Perform calculations
- **ROUND():** Round numeric values to a specified number of decimal places.
- **LIMIT:**
- **GROUP BY:**
- **HAVING:**
- **JOIN'S:**
- **Subqueries:**

Description

Retrieves data from one or more tables.

Filters records based on specified conditions.

Returns only unique values from a column.

Combines multiple conditions in a query.

Checks whether a value exists in a given list.

Filters values within a specified range.

Searches for data matching a specific pattern.

Sorts the result in ascending or descending order.

Perform calculations

Round numeric values to a specified number of decimal places.

Limits the number of rows returned.

Group rows with the same values.

Filter grouped data after GROUP BY.

Combines rows from two or more tables.

Use a query inside another query.

SOLUTION

STUDENT ;

OUTPUT

[illegible]

SELECT

Q2 : DISPLAY TEACHERS TABLE .

SOLUTION

SELECT

FROM

TEACHERS ;

OUTPUT

Teacher_ID	Teacher_Name	Subject	Salary	Experience
201	Rajesh	SQL	50000	5
202	Sunita	Python	60000	8
203	Anil	Power BI	55000	6
204	Meena	Excel	45000	4
NULL	NULL	NULL	NULL	NULL

SELECT

Q3 : DISPLAY COURSES TABLE.

SOLUTION

SELECT

*

FROM

COURSES ;

OUTPUT

Course_ID	Course_Name	Fees
1	SQL	12000
2	Python	25000
3	Power BI	10000
4	Excel	5000

SELECT

Q4 : DISPLAY STUDNET_NAME AND MARKS FROM STUDENT TABLE .

SOLUTION

```
SELECT
STUDENT_NAME , MARKS
FROM
STUDENT ;
```

OUTPUT

STUDENT_NAME	MARKS
Amit	85
Priya	92
Rahul	78
Neha	88
Karan	75
Riya	95
Mohit	69
Anjali	81

SELECT

Q5 : DISPLAY TEACHER_NAME AND SUBJECT FROM TEACHERS TABLE.

SOLUTION

```
SELECT  
TEACHER_NAME , SUBJECT  
FROM  
TEACHERS ;
```

OUTPUT

TEACHER_NAME	SUBJECT
Rajesh	SQL
Sunita	Python
Anil	Power BI
Meena	Excel

SELECT

Q6 : DISPLAY COURSE_NAME AND FEES FROM COURSES TABLE.

SOLUTION

```
SELECT
COURSE_NAME , FEES
FROM
COURSES ;
```

OUTPUT

Course_Name	Fees
SQL	12000
Python	25000
Power BI	10000
Excel	5000

WHERE



Q7 : FIND STUDENT FROM DELHI.

SOLUTION

```
SELECT
STUDENT_NAME, CITY
FROM
STUDENT
WHERE CITY = 'DELHI';
```

OUTPUT

STUDENT_NAME	CITY
Amit	Delhi
Neha	Delhi
Mohit	Delhi

WHERE

Q8 : FIND TEACHERS WHOSE SALARY IS GREATER THEN 50,000.

SOLUTION

```
SELECT
TEACHER_NAME, SALARY
FROM
TEACHERS
WHERE SALARY > 50000;
```

OUTPUT

TEACHER_NAME	SALARY
Sunita	60000
Anil	55000

WHERE

Q9 : DISPLAY STUDENTS WITH MARKS GREATER THAN 80.

SOLUTION

```
SELECT
STUDENT_NAME, MARKS
FROM
STUDENT
WHERE MARKS > 80;
```

OUTPUT

STUDENT_NAME	MARKS
Amit	85
Priya	92
Neha	88
Riya	95
Anjali	81

WHERE

Q10 : FIND TEACHERS WITH MORE THEN 5 YEARS OF EXPERIENCE.

SOLUTION

```
SELECT
TEACHER_NAME, EXPERIENCE
FROM
TEACHERS
WHERE EXPERIENCE > 5;
```

OUTPUT

TEACHER_NAME	experience
Sunita	8
Anil	6

WHERE

Q11 : FIND STUDENT WHOSE AGE IS 20.

SOLUTION

```
SELECT
  STUDENT_NAME, AGE
FROM
  STUDENT
WHERE AGE = 20;
```

OUTPUT

STUDENT_NAME	AGE
Amit	20
Neha	20
Mohit	20

WHERE

Q12 : FIND TEACHER WHO TEACHE 'SQL'.

SOLUTION

```
SELECT
TEACHER_NAME, SUBJECT
FROM
TEACHERS
WHERE SUBJECT = 'SQL';
```

OUTPUT

TEACHER_NAME	SUBJECT
Rajesh	SQL

DISTINCT



Q13: DISPLAY ALL UNIQUE CITIES.

SOLUTION

```
SELECT  
DISTINCT CITY  
FROM  
STUDENT ;
```

OUTPUT

CITY
Delhi
Mumbai
Jaipur
Pune

DISTINCT

Q14 : DISPLAY ALL UNIQUE COURSE NAME.

SOLUTION

```
SELECT  
DISTINCT COURSE_NAME  
FROM  
COURSES ;
```

OUTPUT

COURSE_NAME

SQL

Python

Power BI

Excel

AND / OR



Q15 : FIND STUDENT WHOSE MARKS ARE GREATER THAN 80 AND CITY IS 'DELHI'.

SOLUTION

```
SELECT STUDENT_NAME , MARKS , CITY
      FROM STUDENT
      WHERE MARKS > 80
            AND
            CITY = 'DELHI' ;
```

OUTPUT

STUDENT_NAME	MARKS	CITY
Amit	85	Delhi
Neha	88	Delhi

AND / OR

Q16 : FIND TEACHERS WHOSE SALARY IS GEEATER THAN 50,000 AND EXPERIENCE IS MORE THAN 5 YEARS.

SOLUTION

```
SELECT TEACHER_NAME , SALARY , EXPERIENCE
FROM TEACHERS
WHERE SALARY > 50000
AND
EXPERIENCE > 5 ;
```

OUTPUT

TEACHER_NAME	SALARY	EXPERIENCE
Sunita	60000	8
Anil	55000	6

AND / OR

Q17 : FIND STUDENTS ADMITTED AFTER '2025-02-01' AND MARKS ARE GREATER THAN 75.

SOLUTION

```
SELECT STUDENT_NAME,ADMISSION_DATE,MARKS
      FROM STUDENT
      WHERE MARKS > 75
      AND
      ADMISSION_DATE > '2025-02-01'
      ORDER BY ADMISSION_DATE ASC , MARKS DESC;
```

OUTPUT

STUDENT_NAME	Admission_Date	MARKS
Priya	2025-02-15	92
Riya	2025-02-25	95
Rahul	2025-03-01	78
Anjali	2025-05-05	81

AND / OR

Q18 : FIND STUDENTS FROM DELHI OR JAIPUR.

SOLUTION

```
SELECT STUDENT_NAME , CITY
FROM STUDENT
WHERE CITY = 'DELHI'
OR
      'JAIPUR' ;
```

OUTPUT

STUDENT_NAME	CITY
Amit	Delhi
Neha	Delhi
Mohit	Delhi

AND / OR

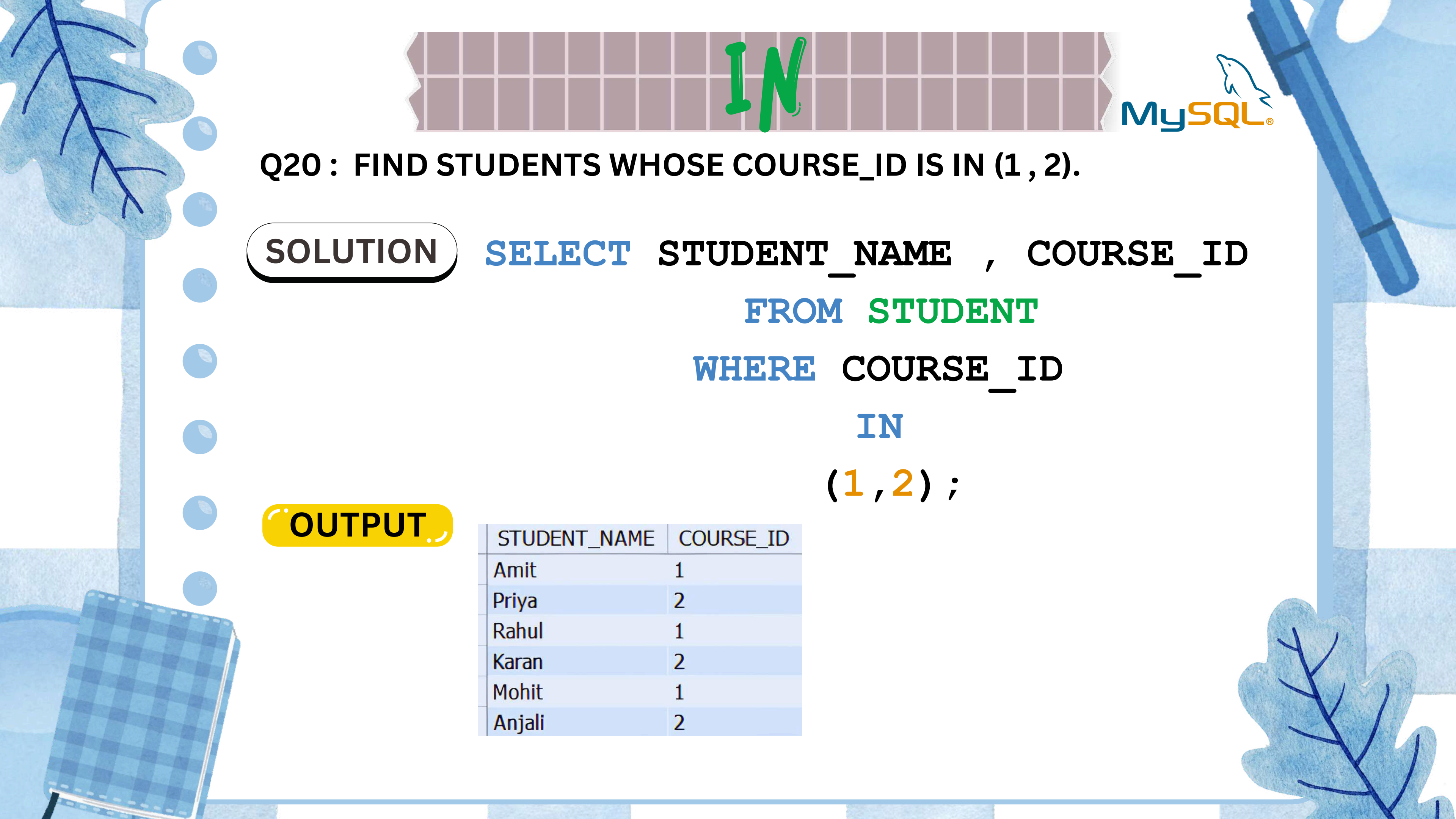
Q19 : FIND TEACHERS WHOSE SUBJECT IS 'SQL' OR 'PYTHON'.

SOLUTION

```
SELECT TEACHER_NAME , SUBJECT
FROM TEACHERS
WHERE SUBJECT = 'SQL'
OR
'PYTHON' ;
```

OUTPUT

TEACHER_NAME	SUBJECT
Rajesh	SQL



IN



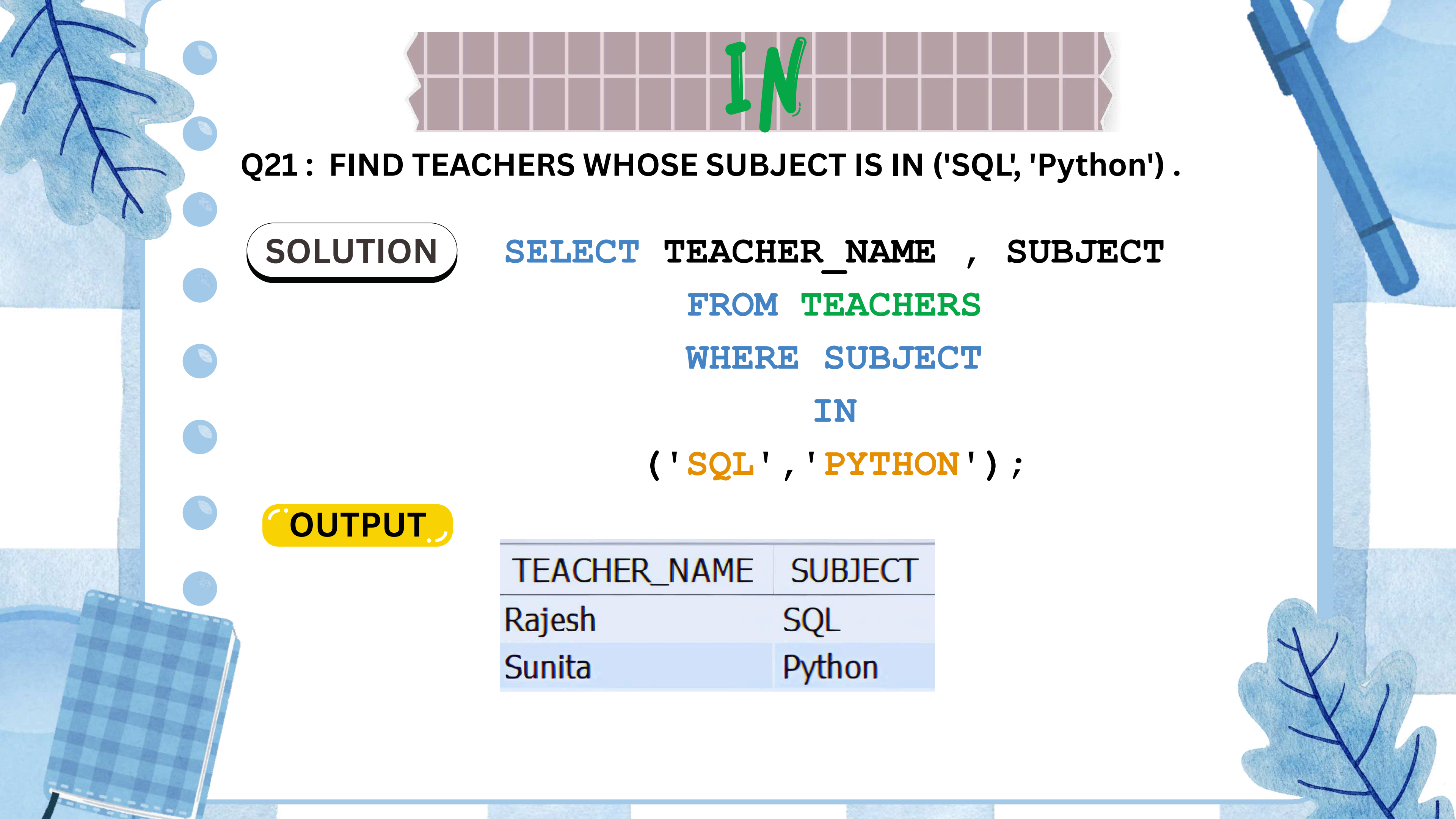
Q20 : FIND STUDENTS WHOSE COURSE_ID IS IN (1 , 2).

SOLUTION

```
SELECT STUDENT_NAME , COURSE_ID  
FROM STUDENT  
WHERE COURSE_ID  
IN  
(1 , 2) ;
```

OUTPUT

STUDENT_NAME	COURSE_ID
Amit	1
Priya	2
Rahul	1
Karan	2
Mohit	1
Anjali	2



IN

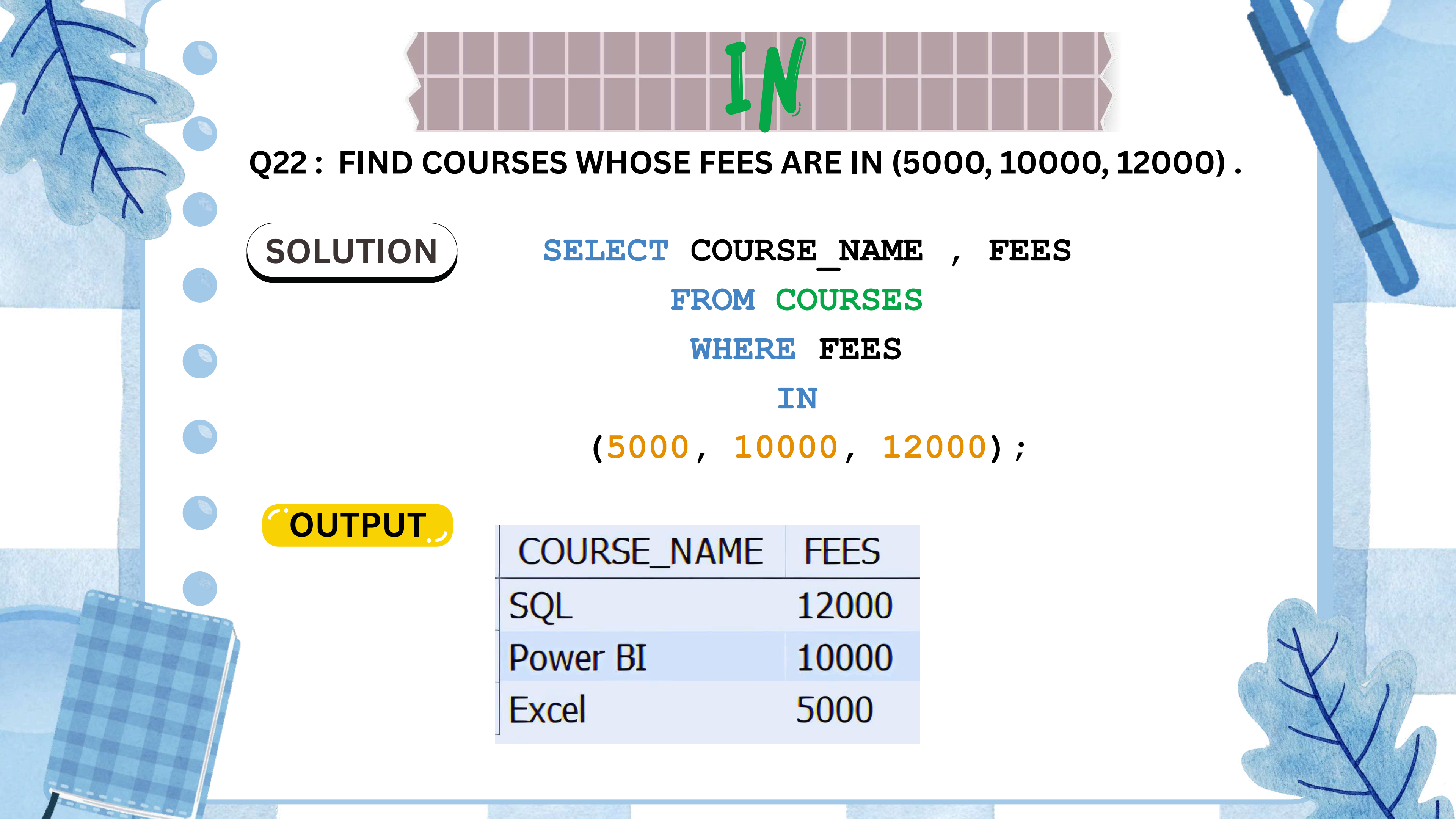
Q21 : FIND TEACHERS WHOSE SUBJECT IS IN ('SQL', 'Python').

SOLUTION

```
SELECT TEACHER_NAME , SUBJECT
FROM TEACHERS
WHERE SUBJECT
IN
('SQL' , 'PYTHON' ) ;
```

OUTPUT

TEACHER_NAME	SUBJECT
Rajesh	SQL
Sunita	Python



IN

Q22 : FIND COURSES WHOSE FEES ARE IN (5000, 10000, 12000) .

SOLUTION

```
SELECT COURSE_NAME , FEES
FROM COURSES
WHERE FEES
IN
(5000 , 10000 , 12000) ;
```

OUTPUT

COURSE_NAME	FEES
SQL	12000
Power BI	10000
Excel	5000

BETWEEN



Q23 : FIND STUDENTS WHOSE MARKS ARE BETWEEN 70 AND 90 .

SOLUTION

```
SELECT STUDENT_NAME , MARKS
FROM STUDENT
WHERE MARKS
BETWEEN
70 AND 90 ;
```

OUTPUT

STUDENT_NAME	MARKS
Amit	85
Rahul	78
Neha	88
Karan	75
Anjali	81

BETWEEN

Q24 : FIND STUDENTS ADMITTED BETWEEN '2025-01-01' AND '2025-03-31' .

SOLUTION

```
SELECT STUDENT_NAME , ADMISSION_DATE  
FROM STUDENT  
WHERE ADMISSION_DATE  
BETWEEN  
'2025-01-01' AND '2025-03-31' ;
```

OUTPUT

STUDENT_NAME	Admission_Date
Amit	2025-01-10
Priya	2025-02-15
Rahul	2025-03-01
Neha	2025-01-20
Riya	2025-02-25
Mohit	2025-03-18

BETWEEN

Q25 : FIND STUDENTS AGE BETWEEN 18 AND 22.

SOLUTION

```
SELECT STUDENT_NAME, AGE
FROM STUDENT
WHERE AGE
      BETWEEN
      18 AND 22 ;
```

OUTPUT

STUDENT_NAME	AGE
Amit	20
Priya	21
Rahul	19
Neha	20
Karan	22
Riya	21
Mohit	20
Anjali	22

LIKE



Q26 : FIND STUDENTS WHOSE NAME START WITH 'A'.

SOLUTION

```
SELECT STUDENT_NAME  
FROM STUDENT  
WHERE STUDENT_NAME  
LIKE  
    'A%';
```

OUTPUT

STUDENT_NAME
Amit
Anjali

LIKE

Q27 : FIND STUDENTS WHOSE NAME END WITH 'A'.

SOLUTION

```
SELECT STUDENT_NAME  
FROM STUDENT  
WHERE STUDENT_NAME  
LIKE  
'%A';
```

OUTPUT

STUDENT_NAME
Priya
Neha
Riya

LIKE

Q28: FIND STUDENTS WHOSE NAME CONTAIN 'A'.

SOLUTION

```
SELECT STUDENT_NAME  
FROM STUDENT  
WHERE STUDENT_NAME  
LIKE  
' %A% ' ;
```

OUTPUT

STUDENT_NAME
Amit
Priya
Rahul
Neha
Karan
Riya
Anjali

ORDER BY



Q29 : SHOW STUDENTS ORDERED BY AGE.

SOLUTION

```
SELECT STUDENT_NAME , AGE  
FROM STUDENT  
ORDER BY AGE ;
```

OUTPUT

STUDENT_NAME	AGE
Rahul	19
Amit	20
Neha	20
Mohit	20
Priya	21
Riya	21
Karan	22
Anjali	22

ORDER BY

Q30 : SHOW TEACHERS ORDERED BY SALARY (DESCENDING).

SOLUTION

```
SELECT TEACHER_NAME , SALARY
FROM TEACHERS
ORDER BY SALARY DESC;
```

OUTPUT

TEACHER_NAME	SALARY
Sunita	60000
Anil	55000
Rajesh	50000
Meena	45000

ORDER BY

Q31 : SHOW STUDENTS ORDERED BY ADMISSION DATE.

SOLUTION

```
SELECT STUDENT_NAME , ADMISSION_DATE  
FROM STUDENT  
ORDER BY ADMISSION_DATE;
```

OUTPUT

STUDENT_NAME	admission_date
Amit	2025-01-10
Neha	2025-01-20
Priya	2025-02-15
Riya	2025-02-25
Rahul	2025-03-01
Mohit	2025-03-18
Karan	2025-04-12
Anjali	2025-05-05

AGGREGATE FUNCTION

Q32 : FIND THE HIGHEST MARKS.



SOLUTION

```
SELECT MAX(MARKS)
FROM
STUDENT ;
```

OUTPUT

MAX(MARKS)
95

AGGREGATE FUNCTION

Q33: FIND THE LOWEST MARKS.

SOLUTION

```
SELECT MIN(MARKS)
FROM
STUDENT;
```

OUTPUT

MIN(MARKS)
69

AGGREGATE FUNCTION

Q34 : FIND THE AVERAGE MARKS.

SOLUTION

```
SELECT AVG (MARKS)
FROM
STUDENT ;
```

OUTPUT

AVG(MARKS)
82.8750

AGGREGATE FUNCTION

Q35 : COUNT TOTAL STUDENTS.

SOLUTION

```
SELECT COUNT (*)  
FROM  
STUDENT ;
```

OUTPUT

COUNT(*)
8

AGGREGATE FUNCTION

Q36 : COUNT TOTAL TEACHERS.

SOLUTION

```
SELECT COUNT (*)  
FROM  
TEACHERS ;
```

OUTPUT

COUNT(*)
4

AGGREGATE FUNCTION

Q37 : FIND TOTAL SALARY OF ALL TEACHERS.

SOLUTION

```
SELECT SUM(SALARY)
FROM
TEACHERS ;
```

OUTPUT

SUM(SALARY)
210000

AGGREGATE FUNCTION

Q38 : FIND MINIMUM SALARY OF TEACHER.

SOLUTION

```
SELECT MIN (SALARY)
FROM
TEACHERS ;
```

OUTPUT

MIN(SALARY)
45000

ROUND



Q39: ROUND AVERAGE STUDENTS MARKS TO 2 DECIMAL PLACES.

SOLUTION

```
SELECT ROUND (AVG (MARKS) ,2) AS STUDENT_AVG  
FROM STUDENT;
```

OUTPUT

STUDENT_AVG
82.88

ROUND

Q40 : ROUND THE AVERAGE TEACHER SALARY TO THE NEAREST INTEGER.

SOLUTION

```
SELECT ROUND (AVG (SALARY) , 0) AS AVG_SALARY  
FROM TEACHER;
```

OUTPUT

AVG_SALARY
52500

LIMIT



Q41 : DISPLAY THE TOP 3 STUDENTS BASED ON MARKS.

SOLUTION

```
SELECT STUDENT_NAME , MARKS
FROM STUDENT
ORDER BY MARKS DESC
LIMIT 3;
```

OUTPUT

STUDENT_NAME	MARKS
Riya	95
Priya	92
Neha	88

LIMIT

Q42 : FIND THE HIGHEST PAID TEACHER .

SOLUTION

```
SELECT TEACHER_NAME AS HIGHEST_PAID_TEACHER , SALARY
FROM TEACHERS
ORDER BY SALARY DESC
LIMIT 1;
```

OUTPUT

highest_paid_teacher	SALARY
Sunita	60000

GROUP BY



Q43 : COUNT STUDENTS IN EACH COURSES.

SOLUTION

```
SELECT COUNT(*) , COURSE_NAME
      FROM STUDENT
      JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
   GROUP BY COURSE_NAME;
```

OUTPUT

COUNT(*)	COURSE_NAME
3	SQL
3	Python
2	Power BI

GROUP BY

Q44 : FIND THE MAXIMUM MARKS IN EACH CITY.

SOLUTION

```
SELECT MAX(MARKS) , CITY  
FROM STUDENT  
GROUP BY CITY ;
```

OUTPUT

MAX(MARKS)	CITY
88	Delhi
92	Mumbai
95	Jaipur
75	Pune

GROUP BY

Q45 : FIND THE AVERAGE AGE OF STUDENTS IN EACH CITY.

SOLUTION

```
SELECT  AVG (AGE)  , CITY
        FROM STUDENT
        GROUP BY CITY;
```

OUTPUT

AVG(AGE)	CITY
20.0000	Delhi
21.5000	Mumbai
20.0000	Jaipur
22.0000	Pune

GROUP BY

Q46 : COUNT MALE AND FEMALE STUDENTS SEPARATELY.

SOLUTION

```
SELECT GENDER , COUNT (*)  
FROM STUDENT  
GROUP BY GENDER;
```

OUTPUT

GENDER	COUNT(*)
Male	4
Female	4

GROUP BY

Q47 : FIND THE TOTAL NUMBER OF STUDENTS IN EACH CITY.

SOLUTION

```
SELECT COUNT (STUDENT_ID) , CITY  
      FROM STUDENT  
      GROUP BY CITY ;
```

OUTPUT

COUNT(STUDENT_ID)	CITY
3	Delhi
2	Mumbai
2	Jaipur
1	Pune

GROUP BY

Q48 : FIND THE AVG MARKS OF MALE AND FEMALE STUDENTS SEPARATELY.

SOLUTION

```
SELECT AVG (MARKS) , GENDER , COUNT (*)  
FROM STUDENT  
GROUP BY GENDER;
```

OUTPUT

AVG(MARKS)	GENDER	COUNT(*)
76.7500	Male	4
89.0000	Female	4

HAVING



Q49 : DISPLAY CITIES HAVING MORE THAN 3 STUDENTS.

SOLUTION

```
SELECT CITY , COUNT (*)  
FROM STUDENT  
GROUP BY CITY  
HAVING COUNT (*) > 2;
```

OUTPUT

CITY	COUNT(*)
Delhi	3

HAVING

Q50 : DISPLAY COURSES HAVING MORE THAN 2 STUDENTS.

SOLUTION

```
SELECT COURSE_NAME ,COUNT(*) AS NUMBER_OF_STUDENT
FROM STUDENT
JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
GROUP BY COURSE_NAME
HAVING COUNT(*) > 2;
```

OUTPUT

COURSE_NAME	NUMBER_OF_STUDENT
SQL	3
Python	3

HAVING

Q51 : SHOW COURSES WHOSE AVG MARKS ARE GREATER THAN 80.

SOLUTION

```
SELECT COURSE_NAME , ROUND (AVG (MARKS) , 2) AS AVG_MARKS
FROM STUDENT
JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
GROUP BY COURSE_NAME
HAVING AVG (MARKS) > 80 ;
```

OUTPUT

COURSE_NAME	AVG_MARKS
Python	82.67
Power BI	91.50

JOIN'S



Q52 : DISPLAY THE STUDENT NAME ALONG WITH THE COURSE NAME IN WHICH EACH STUDENT IS ENROLLED .

SOLUTION

```
SELECT STUDENT_NAME , COURSE_NAME
      FROM STUDENT
      INNER JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
ORDER BY STUDENT_NAME ASC;
```

OUTPUT

STUDENT_NAME	COURSE_NAME
Amit	SQL
Anjali	Python
Karan	Python
Mohit	SQL
Neha	Power BI
Priya	Python
Rahul	SQL
Riva	Power BI

JOIN'S

Q53 : DISPLAY ALL COURSES ALONG WITH THE STUDENTS ENROLLED IN THEM , INCLUDING COURSES THAT HAVE NO STUDENTS .

SOLUTION

```
SELECT STUDENT_NAME , COURSE_NAME
      FROM STUDENT
      LEFT JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
ORDER BY STUDENT_NAME ASC;
```

OUTPUT

STUDENT_NAME	COURSE_NAME
Amit	SQL
Anjali	Python
Karan	Python
Mohit	SQL
Neha	Power BI
Priya	Python
Rahul	SQL
Riva	Power BI

JOIN'S

Q54 : DISPLAY ALL COURSES ALONG WITH THE STUDENTS ENROLLED IN THEM, INCLUDING COURSES THAT HAVE NO STUDENTS .

SOLUTION

```
SELECT STUDENT_NAME , COURSE_NAME
      FROM STUDENT
      RIGHT JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
ORDER BY STUDENT_NAME ASC;
```

OUTPUT

STUDENT_NAME	COURSE_NAME
	Excel
Amit	SQL
Anjali	Python
Karan	Python
Mohit	SQL
Neha	Power BI
Priya	Python
Rahul	SQL
Riya	Power BI

JOIN'S

Q55 : DISPLAY ALL STUDENTS AND ALL COURSES INCLUDING UNMATCHED RECORDS FROM BOTH TABLES .

SOLUTION

```
SELECT STUDENT_NAME , COURSE_NAME
      FROM STUDENT
      LEFT JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
      UNION
      SELECT STUDENT_NAME , COURSE_NAME
      FROM STUDENT
      RIGHT JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
```

OUTPUT

STUDENT_NAME	COURSE_NAME
Amit	SQL
Anjali	Python
Karan	Python
Mohit	SQL
Neha	Power BI
Priya	Python
Rahul	SQL
Riva	Power BI

JOIN'S

Q56 : COUNT STUDENTS FROM EACH TEACHER .

SOLUTION

```
SELECT TEACHER_NAME , COUNT (STUDENT_ID) AS TOTAL_STUDENT
      FROM STUDENT
      INNER JOIN TEACHERS
ON STUDENT.TEACHER_ID = TEACHERS.TEACHER_ID
   GROUP BY TEACHER_NAME ;
```

OUTPUT

TEACHER_NAME	TOTAL_STUDENT
Rajesh	3
Sunita	3
Anil	2

JOIN'S

Q57 : FIND THE LOWEST MARKS IN EACH COURSE .

SOLUTION

```
SELECT MIN(MARKS) , COURSE_NAME
      FROM STUDENT
      INNER JOIN COURSES
ON STUDENT.COURSE_ID = COURSES.COURSE_ID
GROUP BY COURSE_NAME ;
```

OUTPUT

MIN(MARKS)	COURSE_NAME
69	SQL
75	Python
88	Power BI

SUBQUERIES



Q58 : FIND STUDENTS WHO SCORED ABOVE THE AVG MARKS .

SOLUTION

```
SELECT STUDENT_NAME , AVG(MARKS)
      FROM STUDENT
WHERE MARKS > (SELECT AVG(MARKS)
               FROM STUDENT)
GROUP BY STUDENT_NAME ;
```

OUTPUT

STUDENT_NAME	AVG(MARKS)
Amit	85.0000
Priya	92.0000
Neha	88.0000
Riya	95.0000

SUBQUERIES

Q59 : FIND STUDENTS WHOSE MARKS ARE GREATER THAN AMIT MARKS .

SOLUTION

SOLUTION

```
SELECT *  
FROM STUDENT  
WHERE MARKS > (SELECT MARKS FROM STUDENT  
                WHERE STUDENT NAME = 'AMIT' );
```

[illegible]

SUBQUERIES

Q60 : FIND STUDENTS ENROLLED IN THE SAME COURSE AS PRIYA .

SOLUTION

```
SELECT STUDENT_NAME , CLASS , MARKS , COURSE_NAME
      FROM STUDENT
      JOIN COURSES
      ON STUDENT.COURSE_ID = COURSES.COURSE_ID
WHERE STUDENT.COURSE_ID = (SELECT STUDENT.COURSE_ID
                          FROM STUDENT
                          WHERE STUDENT_NAME = 'PRIYA') ;
```

OUTPUT

STUDENT_NAME	CLASS	MARKS	COURSE_NAME
Priya	XII	92	Python
Karan	XII	75	Python
Anjali	XII	81	Python

SUBQUERIES

Q61 : FIND STUDENTS WHOSE MARKS ARE LOWER THAN THE HIGHEST MARKS .

SOLUTION

```
SELECT STUDENT_NAME , MARKS
      FROM STUDENT
 WHERE MARKS < (SELECT MAX(MARKS) FROM STUDENT)
      ORDER BY MARKS DESC;
```

OUTPUT

STUDENT_NAME	MARKS
Priya	92
Neha	88
Amit	85
Anjali	81
Rahul	78
Karan	75
Mohit	69

SUBQUERIES

Q62 : FIND TEACHERS WHOSE SALARY IS HIGHER THAN THE AVG SALARY .

SOLUTION

```
SELECT TEACHER_NAME , SALARY
      FROM TEACHERS
 WHERE SALARY > (SELECT AVG (SALARY) FROM TEACHERS) ;
```

OUTPUT

TEACHER_NAME	SALARY
Sunita	60000
Anil	55000

SUBQUERIES

Q63 : FIND STUDENTS WHO SCORED THE SECOND HIGHEST MARKS .

SOLUTION

```
SELECT STUDENT_NAME , MARKS
FROM STUDENT
WHERE MARKS = (SELECT MAX(MARKS) FROM STUDENT
WHERE MARKS < (SELECT MAX(MARKS) FROM STUDENT)
);
```

OUTPUT

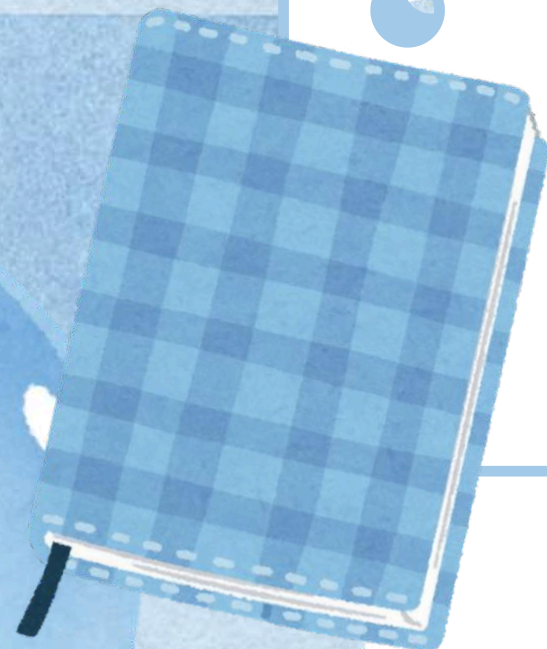
STUDENT_NAME	MARKS
Priya	92



CONCLUSION



The School Management System project demonstrates how SQL can be used to manage and analyze school data efficiently. By using SQL concepts such as SELECT, WHERE, ORDER BY, GROUP BY, Aggregate Functions, JOINS, LIMIT, and Subqueries, the project helps retrieve accurate information and generate meaningful reports. This project also enhanced my SQL skills and practical understanding of database management.





THANK YOU

 Presented by : VASUDEV SHARMA

